MULTI-AGENT STOCK RESEARCH ANALYST

AI-Powered Stock Research Using Specialized Agents, Financial Data Analysis, News Intelligence, and Risk Assessment

1. Title

Multi-Agent Stock Research Analyst: An AI-powered stock research platform that analyzes publicly traded companies using multiple specialized AI agents for fundamental analysis, technical analysis, financial news analysis, and risk assessment. The system combines their findings into a comprehensive research report supported by data, source links, and explanations.

2. Objective

Build a full-stack web application using React.js for the frontend and FastAPI for the backend that allows users to research publicly traded stocks by entering a company name or ticker symbol.

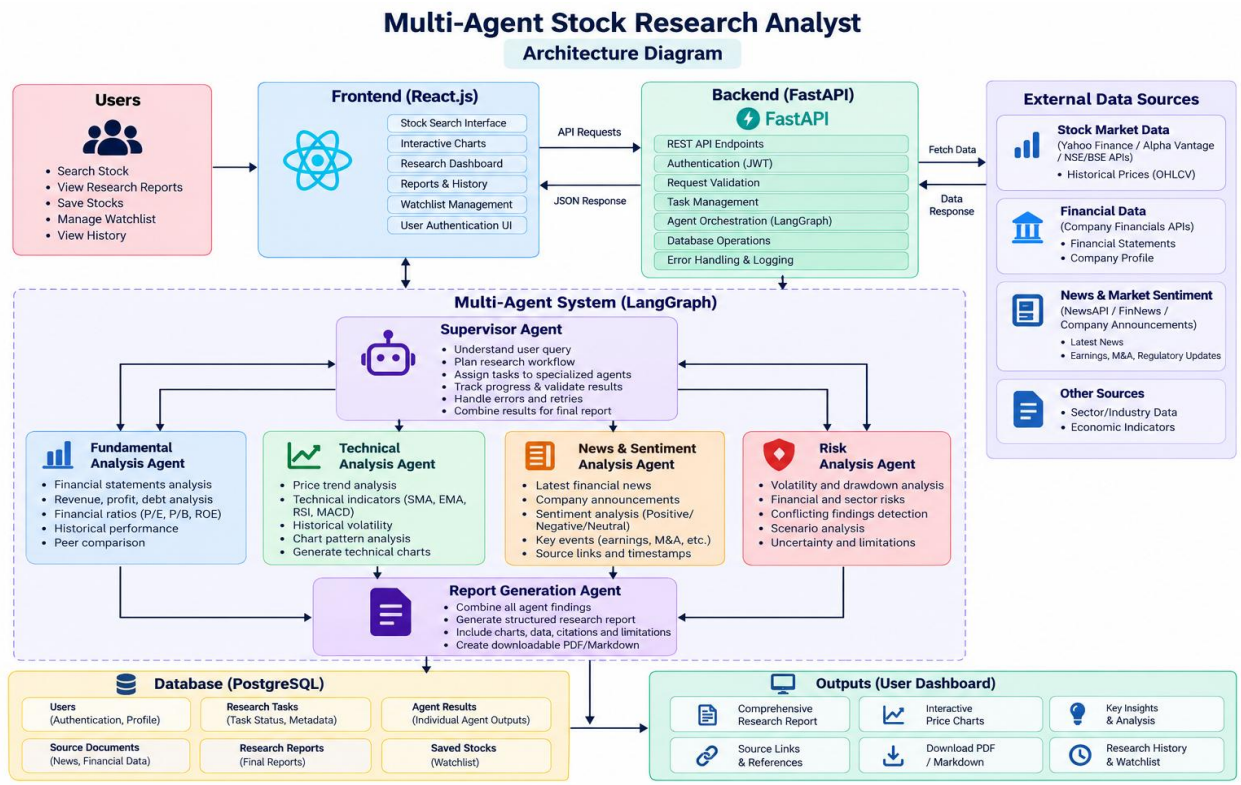
The platform uses a multi-agent architecture in which each AI agent performs a specific research task. A supervisor agent coordinates these agents, collects their findings, checks the completeness of the results, and generates a consolidated stock research report.

Expected Outcomes

- Develop a full-stack React.js and FastAPI application.
- Implement a multi-agent architecture using LangGraph.
- Retrieve historical stock prices and available company financial information.
- Build a Fundamental Analysis Agent to analyze company financial performance.
- Build a Technical Analysis Agent to calculate indicators and examine price trends.
- Build a News and Sentiment Agent to analyze relevant financial news and company announcements.
- Build a Risk Analysis Agent to identify potential risks and conflicting evidence.
- Implement a Supervisor Agent to coordinate the research workflow.
- Generate comprehensive stock research reports with source links and timestamps.
- Create an interactive dashboard with stock charts, analysis results, and research history.
- Store research tasks, agent results, and reports in PostgreSQL.
- Evaluate the accuracy of financial calculations, citation quality, and overall workflow reliability.

3. Architecture Diagram

The proposed system consists of the following components:



4. Modules to be Implemented

1. User Authentication and Research Workspace

- User registration and login.
- JWT-based authentication.
- User-specific research history.
- Saved stocks and watchlists.
- User preferences, including market and analysis period.

2. Stock Search and Market Data Collection

- Search for companies using their name or ticker symbol.
- Retrieve historical stock prices.
- Collect OHLCV data: Open, High, Low, Close, and Volume.
- Retrieve available company profiles and financial statements.
- Record data sources and retrieval timestamps.
- Validate missing values, duplicate dates, and inconsistent data.
- Support the selected stock market, such as NSE/BSE or NYSE/Nasdaq.

3. Fundamental Analysis Agent

This agent analyzes the financial performance and condition of a company.

Features:

- Revenue and earnings analysis.
- Profitability analysis.
- Debt and financial stability analysis.
- Calculation of financial ratios such as P/E, P/B, and ROE when the required data is available.
- Historical financial performance comparison.
- Peer comparison where comparable data is available.
- Identification of missing or inconsistent financial information.

Example: When a user searches for a company, the agent retrieves available financial data and explains its revenue growth, profitability, debt levels, and relevant valuation metrics.

4. Technical Analysis Agent

This agent analyzes historical stock prices and market trends.

Features:

- Historical price trend analysis.
- Simple Moving Average (SMA).
- Exponential Moving Average (EMA).
- Relative Strength Index (RSI).
- Moving Average Convergence Divergence (MACD).
- Historical volatility calculation.
- Price and indicator charts.
- Identification of predefined technical conditions.

Example: The agent examines a stock's historical price data and explains whether its selected technical indicators show an upward trend, downward trend, or mixed signals.

Technical indicators describe historical market conditions and should not be treated as guaranteed predictions.

5. News and Sentiment Analysis Agent

This agent analyzes financial news and company announcements relevant to the selected stock.

Features:

- Retrieve recent financial news from permitted sources.
- Analyze official company announcements.
- Summarize important corporate events.

- Identify earnings announcements, acquisitions, and regulatory developments.
- Classify news sentiment as positive, negative, or neutral.
- Attach source links and publication dates.
- Identify conflicting reports and uncertain information.

Example: If a company announces an acquisition, the agent summarizes the announcement, identifies the relevant source, and explains why the event may matter to investors.

6. Risk Analysis Agent

This agent examines potential risks and evaluates the reliability of the combined research findings.

Features:

- Historical volatility analysis.
- Maximum drawdown calculation.
- Identification of financial and sector-related risks.
- Detection of conflicting findings across agents.
- Identification of missing or outdated information.
- Analysis of alternative scenarios.
- Explanation of uncertainty and limitations.

Example: If the technical analysis looks positive but the fundamental analysis identifies declining profitability, the Risk Agent highlights the disagreement rather than ignoring it.

7. Supervisor Agent and Multi-Agent Orchestration

The Supervisor Agent manages the entire research process using LangGraph.

Features:

- Understand the user's research request.
- Plan the required research steps.
- Assign tasks to specialized agents.
- Track task progress and agent outputs.
- Validate structured results.
- Handle failed data retrieval and incomplete outputs.
- Retry failed steps within configured limits.
- Identify conflicting findings.
- Trigger final report generation when the required analyses are complete.

Example: If the News Agent fails to retrieve relevant information, the supervisor can retry the retrieval or mark the news analysis as unavailable. It should not invent news to fill the gap.

8. AI-Powered Research Report Generation

This module combines the findings of all agents into one structured report.

Features:

- Company overview.
- Fundamental analysis.
- Technical analysis.
- News and sentiment summary.
- Risk assessment.
- Conflicting findings.
- Supporting source links and timestamps.
- Data limitations and uncertainty.
- Downloadable PDF or Markdown reports.

The report should clearly distinguish factual data, calculated metrics, and AI-generated interpretations.

9. Dashboard and Research History

Features:

- Stock search interface.
- Research task status.
- Stock price and indicator charts.
- Fundamental analysis dashboard.
- News summary and source links.
- Risk analysis section.
- Consolidated research report.
- Saved stocks and watchlists.
- Previous research reports.
- PDF or Markdown export.

10. Testing, Integration, and Deployment

- Test all backend API endpoints.
- Validate financial calculations against independent reference calculations.
- Test agent routing and structured outputs.
- Test error handling and retry logic.

- Verify source links and factual citation support.
- Measure task completion rate, response time, and cost per report.
- Protect API keys using environment variables.
- Deploy the frontend, backend, and database.
- Configure HTTPS, CORS, logging, and appropriate access controls.

5. Week-Wise Module Implementation and High-Level Requirements

Milestone 1: Week 1 & 2 — Requirements, Database Design, and Backend Setup

Tasks

- Define the project scope and supported stock market.
- Select the initial stock universe and analysis period.
- Identify suitable financial data and news providers.
- Design the database schema:
    - users
    - research_tasks
    - agent_results
    - source_documents
    - research_reports
    - saved_stocks
- Initialize the FastAPI backend.
- Configure PostgreSQL or SQLite for local development.
- Create the React frontend skeleton.
- Implement stock search and historical price retrieval.
- Validate the retrieved market data.

Outcomes

- Backend and frontend architecture setup completed.
- Database schema finalized.
- Historical market data retrieval operational.
- Basic stock search interface functional.

Milestone 2: Week 3 & 4 — Financial Analysis Tools and Individual Agents

Tasks

- Implement financial ratio calculations.

- Implement technical indicators such as SMA, EMA, RSI, and MACD.
- Build the Fundamental Analysis Agent.
- Build the Technical Analysis Agent.
- Integrate a financial news source.
- Build the News and Sentiment Agent.
- Build the Risk Analysis Agent.
- Define structured inputs and outputs for each agent.
- Validate calculations against independently computed results.

Outcomes

- Individual research agents operational.
- Financial calculations tested.
- Technical charts available.
- News summaries include source links and publication dates.
- Agent outputs follow defined schemas.

Milestone 3: Week 5 & 6 — Supervisor Agent and Report Generation

Tasks

- Implement the Supervisor Agent using LangGraph.
- Define the research workflow and agent routing.
- Implement state management and conditional transitions.
- Add error handling and bounded retries.
- Validate agent outputs and identify conflicting findings.
- Implement the Report Generation Agent.
- Combine fundamental, technical, news, and risk analyses.
- Store research tasks, agent results, and reports in PostgreSQL.
- Test the complete multi-agent workflow.

Outcomes

- Supervisor-based orchestration functional.
- End-to-end stock research workflow operational.
- Consolidated research reports generated.
- Research history stored in the database.
- Source links and data limitations included in reports.

Milestone 4: Week 7 & 8 — Dashboard, Evaluation, and Deployment

Tasks

- Build the research dashboard using React.js.
- Integrate stock search and report-generation APIs.
- Display price charts, financial metrics, news, and risk findings.
- Implement saved stocks and research history.
- Add PDF or Markdown report export.
- Test API reliability and failure recovery.
- Evaluate calculation correctness and citation quality.
- Measure response time and cost per research task.
- Deploy the frontend and backend.
- Configure PostgreSQL, environment variables, HTTPS, and CORS.

Outcomes

- Fully integrated application deployed.
- Dashboard and research history operational.
- Complete end-to-end workflow demonstrable.
- Testing and evaluation results documented.
- Project limitations and future enhancements documented.

6. Evaluation Criteria

Milestone 1 — Week 2

- Backend scaffolding completed.
- Database schema finalized.
- Stock search and historical data retrieval functional.
- Data validation checks implemented.
- Frontend successfully communicates with the backend.

Milestone 2 — Week 4

- Fundamental calculations verified.
- Technical indicators validated.
- Individual agents return structured outputs.
- News summaries include source links and dates.
- Risk metrics are calculated reproducibly.

Milestone 3 — Week 6

- Supervisor routes tasks to the appropriate agents.
- Agent failures are handled correctly.
- Research results are consolidated into a report.
- Reports include source metadata and limitations.
- Research tasks and outputs are persisted successfully.

Milestone 4 — Week 8

- Dashboard supports the complete research workflow.
- Charts and reports display correctly.
- Citation coverage and citation correctness are evaluated.
- Task completion rate and response time are measured.
- End-to-end tests pass.
- Deployment is accessible and functional.

Overall Evaluation Metrics

- Calculation accuracy: Agreement between financial metrics and independently computed reference values.
- Citation coverage: Percentage of factual claims supported by sources.
- Citation correctness: Whether cited sources actually support the claims.
- Agent completion rate: Percentage of tasks completed successfully.
- Error recovery: Ability to handle failed retrievals or incomplete agent outputs.
- Response time: Time required to generate a report.
- Operational cost: Cost per research task.
- Report completeness: Whether all required sections are included.

7. Tools and Tech Stack

Programming Language

- Python

Backend

- FastAPI
- Pydantic

Frontend

- React.js

- JavaScript or TypeScript
- Tailwind CSS
- Plotly for interactive charts

Agentic AI and LLM

- LangGraph for multi-agent orchestration.
- A tool-calling-capable LLM for planning, summarization, and report generation.
- Structured outputs for communication between agents.

Financial Data and Analysis

- A suitable market data API or historical stock dataset.
- Financial news APIs and official company announcements.
- pandas
- NumPy
- scikit-learn, if required for optional analytical extensions.
- Validated Python functions for financial ratios and technical indicators.

Database

- PostgreSQL for the main application.
- SQLite as an optional local-development database.

Authentication

- JWT-based authentication.
- Secure password hashing.

Testing

- pytest
- FastAPI test client.
- API integration tests.
- Numerical validation tests.

Development and Deployment

- VS Code.
- Git and GitHub.
- Docker and Docker Compose.
- GitHub Actions.
- A suitable cloud hosting provider.

- Postman for API testing.

8. Proposed Database Structure

Users

- user_id
- email
- password_hash
- created_at

Research Tasks

- task_id
- user_id
- ticker
- market
- requested_period
- status
- created_at
- completed_at

Agent Results

- result_id
- task_id
- agent_name
- status
- output_json
- error_message
- created_at

Source Documents

- source_id
- task_id
- source_type
- title
- source_url
- published_at

- retrieved_at
- content_hash

Research Reports

- report_id
- task_id
- report_content
- summary
- limitations
- created_at

Saved Stocks

- saved_stock_id
- user_id
- ticker
- created_at